
FLEXIBLE JOB SHOP SCHEDULING PROBLEM WITH TIME WINDOWS

Siwen Liu

School of Information Science and Technology
Sandau University
Shanghai
siwenliu@sandau.edu.cn

June 9, 2025

ABSTRACT

1 Introduction

Community group buying has rapidly gained popularity as an e-commerce model, offering consumers a convenient way to purchase fresh products through online platforms, with the option to pick up items at designated locations the following day. This model streamlines the supply chain by reducing intermediaries, which lowers product prices and improves cost-effectiveness. It has become especially popular among urban consumers in China, addressing the demand for both convenience and affordability. The supply chain in community group buying consists of three main components: warehouses, distributors, and self-pickup points. Warehouses process and sort goods from suppliers according to distributor needs, and ship them to distributors on time. Distributors then perform secondary sorting before delivering goods to self-pickup points for consumer collection. Efficient warehouse operations are critical to meeting tight delivery deadlines and minimizing operational costs. Despite the advantages, the scheduling of warehouse operations presents several challenges. Goods in the warehouse must undergo different processing steps (e.g., shelving, processing, sorting), and resources such as sorting docks and processing lines are limited. Moreover, suppliers deliver goods in multiple batches based on predicted demand, leading to dynamic and unpredictable arrivals. Each distributor also has personalized delivery requirements based on order volume, product type, and the distance to self-pickup points. These complexities make it difficult to optimize warehouse schedules while minimizing production costs and ensuring timely deliveries.

Thus, this research focuses on optimizing warehouse scheduling in community group buying, addressing these challenges by modeling the problem as a flexible job shop scheduling problem (FJSP). The aim is to develop a solution that minimizes cost and maximizes efficiency, considering dynamic deliveries, limited processing resources, and varying delivery requirements.

1.1 Motivation

The rapid emergence of *New Retail*—an innovative commerce model that integrates online platforms with offline fulfillment infrastructures—has significantly transformed traditional supply chains into highly responsive, data-driven ecosystems. In this paradigm, customer demands are collected in real time through online channels and immediately propagated upstream to manufacturing and distribution units. Consequently, production systems are increasingly required to operate under dynamic and uncertain conditions, where job requests do not follow a fixed schedule but instead arrive sporadically and unpredictably in response to volatile consumer behavior.

This shift presents profound challenges to conventional production scheduling methods, which are typically designed for static environments with complete information known in advance. In particular, the emergence of **dynamic job arrivals** necessitates scheduling models that are capable of real-time responsiveness. In community group-buying and on-demand fulfillment platforms—key application scenarios under New Retail—consumer orders are often consolidated

within short time windows and subsequently transmitted to central warehouses or manufacturing centers for rapid execution. These production jobs are then processed and dispatched to geographically distributed distributors, all under tight time constraints and strict delivery commitments.

To meet these requirements, central warehouse operations must simultaneously address two tightly coupled decision-making stages: (i) the scheduling of incoming production jobs on a set of flexible machines, and (ii) the batching and dispatching of completed jobs to external distributors. The complexity of this *Integrated Flexible Job Shop Scheduling and Dispatching Problem with Dynamic Job Arrivals* (IFJSSP-DP-DJA) stems not only from the combinatorial nature of each subproblem, but also from the need to integrate them under real-time, incomplete, and evolving information flows.

Traditional optimization-based scheduling techniques often struggle to provide timely solutions in such dynamic settings due to their computational complexity and lack of adaptability. In contrast, advanced learning-based methods—particularly those grounded in **deep reinforcement learning (DRL)**—offer promising capabilities for reactive decision-making, scalability, and policy generalization in uncertain environments. This motivates the development of novel scheduling frameworks that can continuously learn and adapt under the evolving conditions of New Retail-driven production-distribution systems.

2 Literature Review

3 Problem Description

This study addresses the Integrated Flexible Job Shop Scheduling and Dispatching Problem under Dynamic Job Demand (IFJSSP-DP-DJD), which arises in production environments where a set of jobs arrives dynamically over time. These jobs must be processed and subsequently dispatched to multiple distributors, subject to specific delivery requirements.

The concept of dynamic job demand refers to a scenario where job requests are driven by real-time consumer orders, resulting in unpredictable and time-varying job arrivals. Unlike traditional batch-oriented production systems, where job arrivals are generally predetermined, dynamic job demand introduces significant uncertainty as jobs are continuously generated by customer demands, often with varying processing times and delivery constraints. This situation is particularly prevalent in the context of modern retail business models, where online demand is tightly integrated with offline fulfillment systems. Consumer orders are transmitted in near real-time to upstream production systems, creating a dynamic flow of job requests that are unpredictable throughout the scheduling horizon.

The central challenge of this problem lies in the simultaneous optimization of two interdependent decision stages:

Job Scheduling in a Flexible Job Shop: In the processing stage, each job consists of a predefined sequence of operations. These operations can be processed on multiple alternative machines, with machine-dependent and deterministic processing times. However, the exact arrival times of jobs are not known in advance and are subject to dynamic fluctuations. This necessitates a responsive and adaptive scheduling strategy that can make real-time decisions as new jobs arrive. Key decisions include assigning operations to machines, sequencing operations while adhering to precedence constraints, and ensuring machine exclusivity and capacity limitations.

Job Dispatching and Batching: Upon completion of processing, jobs must be grouped into batches and dispatched to designated distributors. Each batch incurs a fixed base loading time and an additional variable time proportional to its size. A batch can only be dispatched once all jobs within it are fully processed and loaded. Furthermore, each job is uniquely assigned to one batch for its corresponding distributor. Distributors have specific delivery requirements, including a required completion percentage of jobs and a due date by which the batch must be dispatched. Tardiness in meeting these delivery requirements incurs a penalty, with the penalty weight reflecting the importance of each requirement.

The objective of this study is to develop an integrated scheduling and dispatching plan that minimizes the total weighted tardiness of all delivery requirements. The dynamic nature of job demand—driven by the variability in end-user demand and the real-time requirements of order fulfillment—introduces temporal uncertainty into the system. This uncertainty necessitates the development of a flexible, event-driven scheduling framework capable of incrementally updating schedules in real time to ensure timely job fulfillment and efficient resource utilization.

3.1 Mathematical Model

This section details the mathematical model for the integrated flexible job shop scheduling and dispatching problem. The model aims to minimize the total weighted tardiness of deliveries by coordinating processing operations and dispatching activities.

The indices and sets used in the model are defined in Table 1.

Table 1: Indices and Sets

Set Notation	Description
J	Set of all jobs to be processed.
M	Set of all available machines.
O_j	Sequence of operations for job $j \in J$. Let $ O_j $ be the number of operations for job j .
$A_{j,o}$	Set of available machines for operation o of job j .
D	Set of all distributors.
I_d	Set of jobs to be dispatched to distributor $d \in D$.
B_d	Set of available dispatch batches for distributor $d \in D$.
R	Set of all delivery requirements.

The parameters used in the model are defined in Table 2.

Table 2: Parameters

Symbol	Stage	Description
$P_{j,o,m}$	Processing	Processing time of operation o of job j on machine m .
T_{base}	Dispatching	Base loading time for a dispatch batch.
T_{item}	Dispatching	Loading time per item in a dispatch batch.
ρ_r	Dispatching	For delivery requirement $r \in R$: required completion percentage (e.g., 0.9 for 90%).
δ_r	Dispatching	For delivery requirement $r \in R$: due date.
w_r	Dispatching	For delivery requirement $r \in R$: tardiness weight.

The decision variables used in the model are defined in Table 3.

Table 3: Decision Variables

Symbol	Type / Domain	Description
<i>Processing Stage Variables</i>		
$x_{j,o,m}$	Binary $\in \{0, 1\}$	1 if operation o of job j is assigned to machine m ; 0 otherwise.
$s_{j,o}$	Continuous ≥ 0	Start time of operation o of job j .
$e_{j,o}$	Continuous ≥ 0	End time of operation o of job j .
$z_{j,o,j',o',m}$	Binary $\in \{0, 1\}$	(Implicit in notebook, for Big-M) 1 if operation (j, o) precedes (j', o') on machine m ; 0 if (j', o') precedes (j, o) on machine m .
<i>Dispatching Stage Variables</i>		
$y_{i,d,b}$	Binary $\in \{0, 1\}$	1 if job i (assuming $i \in J$) is assigned to batch b for distributor d ; 0 otherwise.
$t_{d,b}$	Continuous ≥ 0	Dispatch time of batch b for distributor d .
τ_r	Continuous ≥ 0	Tardiness for delivery requirement r .
$q_{r,b}$	Binary $\in \{0, 1\}$	1 if batch b is used to fulfill delivery requirement r ; 0 otherwise.

The objective is to minimize the total weighted tardiness of all delivery requirements:

$$\min \sum_{r \in R} w_r \tau_r \quad (1)$$

$$e_{j,o} \leq s_{j,o+1} \quad \forall j \in J, o \in O_j \setminus \{|O_j| - 1\} \quad (2)$$

$$\sum_{m \in A_{j,o}} x_{j,o,m} = 1 \quad \forall j \in J, o \in O_j \quad (3)$$

$$e_{j,o} = s_{j,o} + \sum_{m \in A_{j,o}} P_{j,o,m} \cdot x_{j,o,m} \quad \forall j \in J, o \in O_j \quad (4)$$

$$s_{j',o'} \geq e_{j,o} - K \cdot (3 - x_{j,o,m} - x_{j',o',m} - z_{j,o,j',o',m}) \quad \forall (j,o) \neq (j',o'), m \quad (5)$$

$$s_{j,o} \geq e_{j',o'} - K \cdot (2 - x_{j,o,m} - x_{j',o',m} + z_{j,o,j',o',m}) \quad \forall (j,o) \neq (j',o'), m \quad (6)$$

$$\sum_{b \in B_d} y_{i,d,b} = 1 \quad \forall d \in D, i \in I_d \quad (7)$$

$$\sum_{k \in I_d} y_{k,d,b} \leq |I_d| \cdot u_{d,b} \quad \forall d \in D, b \in B_d \quad (8)$$

$$u_{d,b} \leq \sum_{k \in I_d} y_{k,d,b} \quad \forall d \in D, b \in B_d \quad (9)$$

$$t_{d,b} \geq e_{i,|O_i|-1} + T_{\text{base}} \cdot u_{d,b} + T_{\text{item}} \sum_{k \in I_d} y_{k,d,b} - K(1 - y_{i,d,b}) \quad \forall d \in D, b \in B_d, i \in I_d \quad (10)$$

$$\sum_{b \in B_{d_r}} q_{r,b} = 1 \quad \forall r \in R \quad (11)$$

$$q_{r,b} \leq u_{d_r,b} \quad \forall r \in R, b \in B_{d_r} \quad (12)$$

$$\tau_r \geq t_{d_r,b} - \delta_r - K(1 - q_{r,b}) \quad \forall r \in R, b \in B_{d_r} \quad (13)$$

$$s_{j,o}, e_{j,o} \geq 0 \quad \forall j \in J, o \in O_j \quad (14)$$

$$t_{d,b} \geq 0 \quad \forall d \in D, b \in B_d \quad (15)$$

$$\tau_r \geq 0 \quad \forall r \in R \quad (16)$$

The meaning of each formula is as follows: The first formula is the objective function, which minimizes the total weighted tardiness of all delivery requirements. Constraint (2) ensures that the operations of each job are processed in the specified order, so that the start time of the next operation cannot be earlier than the end time of the previous one. Constraint (3) guarantees that each operation is assigned to exactly one available machine. Constraint (4) defines that the end time of each operation equals its start time plus the processing time on the assigned machine. Constraints (5) and (6) use the Big-M method to prevent any two operations from being processed simultaneously on the same machine, thus ensuring machine capacity. Constraint (7) requires that each job for a distributor is assigned to exactly one dispatch batch. Constraints (8) and (9) linearize the indicator variable for whether a batch is used. Constraint (10) ensures that the dispatch time of a batch is not earlier than the completion time of all jobs assigned to it plus the required loading time. Constraint (11) enforces that each delivery requirement is fulfilled by exactly one batch, and constraint (12) ensures that only used batches can be assigned to delivery requirements, thus establishing a one-to-one correspondence between batches and delivery times. Constraint (13) calculates the tardiness for each delivery requirement based on the dispatch time of the corresponding batch and the due date. Finally, constraints (14), (15), and (16) enforce the non-negativity of all time-related variables to ensure the physical feasibility of the model.

3.2 Theoretical Support for Scheduling Policy Design

In this section, we present theoretical results that support the design of reward functions and prioritization strategies in the proposed HDRL scheduling framework. These lemmas are particularly relevant for Chapter 3, where the Integrated Flexible Job Shop Scheduling and Dispatching Problem (IFJSSP-DP) is formulated. The following results provide insights into task sequencing and batch dispatch strategies that help accelerate learning and improve system performance.

Additionally, we consider an extension to the original problem formulation by incorporating uncertainty in job processing times. Specifically, we assume that the actual processing time of each operation is a random variable whose value is not known until the operation begins. This extension reflects practical scenarios such as human-performed tasks,

unpredictable delays in machinery, or variability in product handling, and introduces an additional layer of complexity to the scheduling environment.

In the updated problem setting, let the processing time $P_{j,o,m}$ of operation o for job j on machine m be a random variable, denoted as $\tilde{P}_{j,o,m}$. The realization of $\tilde{P}_{j,o,m}$ is unknown until the operation is initiated. Consequently, the scheduler must make dispatching and sequencing decisions based on historical estimates, expected values $\mathbb{E}[\tilde{P}_{j,o,m}]$, or sampling-based predictions, rather than deterministic values.

This uncertainty affects both stages of the IFJSSP-DP:

- In the processing stage, machine allocation and sequencing must be robust to variability in execution times, potentially relying on adaptive policies that update as real-time feedback becomes available.
- In the dispatching stage, the uncertainty in completion times propagates to uncertainty in dispatch timing, affecting batch formation and due-date compliance. The reward design in HDRL must therefore penalize both high variance and systematic delay.

The following theoretical results still apply in expectation, and guide how the HDRL framework can prioritize low-risk (low-variance) actions when optimizing under uncertainty.

Lemma 1. *Let $\mathcal{J}' = \{J_1, J_2, \dots, J_n\}$ be a set of n jobs ready for processing on a single shared machine, with random processing times $\tilde{p}_j$ having known expectations $\mu_j = \mathbb{E}[\tilde{p}_j]$. Let w_j be the weight associated with job J_j . Then, scheduling the jobs in ascending order of μ_j minimizes the expected total weighted waiting time:*

$$\mathbb{E}[\text{Total Waiting Time}] = \sum_{j=1}^n w_j \cdot \mathbb{E}[W_j]$$

where W_j is the waiting time experienced by job J_j .

Proof. Let π be a job sequence and $W_j^{(\pi)}$ the waiting time for job J_j under π . Since waiting times depend linearly on the sum of preceding job durations, we have:

$$\mathbb{E}[W_j^{(\pi)}] = \sum_{k=1}^{j-1} \mu_{\pi(k)}$$

Thus the total expected weighted waiting time is:

$$\mathbb{E} \left[\sum_{j=1}^n w_j W_j^{(\pi)} \right] = \sum_{j=1}^n w_j \sum_{k=1}^{j-1} \mu_{\pi(k)} = \sum_{1 \leq k < j \leq n} w_j \mu_{\pi(k)}$$

To minimize this sum, we arrange μ_j in ascending order. The proof parallels the deterministic SPT case, extended to expected values. $\square$

Lemma 2. *Let batch b serve delivery requirement r with dispatch time t_b and due date δ_r . Let ρ_r be the required delivery percentage for requirement r , and w_r be the associated tardiness penalty weight. The penalty incurred for tardiness is:*

$$L_b = w_r \cdot \rho_r \cdot \max(0, t_b - \delta_r)$$

Then, L_b is monotonically increasing with ρ_r , assuming all other variables are fixed.

Proof. We analyze the derivative of L_b with respect to ρ_r . Define $\Delta = \max(0, t_b - \delta_r)$, which is non-negative by definition. Then:

$$\frac{\partial L_b}{\partial \rho_r} = w_r \cdot \Delta$$

Since $w_r > 0$ and $\Delta \geq 0$, we conclude that L_b is monotonically increasing with ρ_r . Hence, batches fulfilling denser delivery requirements incur higher potential penalty when delayed. This result justifies reward design strategies that prioritize jobs with high-density delivery constraints. $\square$

Proposition 1. *In the low-level policy learning process of HDRL, incorporating priority-based subgoals (e.g., tagging urgent jobs or vehicles) leads to faster convergence of the policy network under sparse reward conditions.*

Proof. Let π_{std} be a low-level policy trained with standard reward, and π_{shaped} a policy trained with additional subgoal shaping signals for priority tasks. Under sparse reward settings, gradient signals $\nabla J(\theta)$ computed via temporal difference (TD) updates are weak, leading to slow updates in π_{std} .

When priority tasks are tagged and rewarded (shaped), we effectively increase the frequency of reward events, improving the temporal credit assignment. This results in stronger TD error signals, leading to larger and more informative gradient updates:

$$\nabla J(\theta) \propto \mathbb{E}[\delta_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

With reward shaping, δ_t is larger in magnitude and more frequent, thus accelerating convergence. Empirical studies in curriculum learning and reward shaping reinforce this theoretical insight. $\square$

These theoretical results validate the structure of the hierarchical reinforcement learning framework proposed in Chapter 4, and justify key design choices in reward shaping, scheduling prioritization, and dispatch coordination.

4 Solution Methodology

To address the integrated warehousing and dispatching problem under dynamic job demand, three core challenges must be jointly solved: (1) determining when to consolidate and respond to newly arriving demands, (2) efficiently scheduling jobs within a flexible job shop environment with multiple machines and operations, and (3) forming dispatching batches for completed jobs in a timely and cost-effective manner. These challenges are inherently hierarchical in nature and span different temporal and decision-making scales. Motivated by this structure, we propose a Hierarchical Deep Reinforcement Learning (HDRL) framework composed of a high-level *Meta-Controller*, responsible for strategic order release and prioritization, and two specialized low-level actor-critic policies, one for dynamic job scheduling and the other for dispatch batch formation. This layered design allows the system to coordinate long-term delivery goals with short-term production efficiency in a principled and scalable manner.

4.1 High-Level Agent: Meta-Controller Design

To address long-term production and delivery efficiency in the presence of dynamic job arrivals and fluctuating workloads, the high-level agent, referred to as the Meta-Controller, is formulated as a reinforcement learning agent operating on aggregated system states. The Meta-Controller is responsible for strategic decisions, including the timing of job release from the input buffer, the setting of order priority levels, and the triggering of finished order dispatch. It learns a policy to maximize delivery efficiency while maintaining balanced machine workloads.

Formally, the high-level decision-making process is modeled as a Markov Decision Process $\mathcal{M}^H = (S^H, A^H, T^H, R^H, \gamma^H)$.

The high-level agent requires a global perspective of the system to make strategic release and dispatching decisions. Therefore, the high-level state S_t^H is constructed as:

$$S_t^H = [\bar{q}_t, \sigma_t^{\text{mach}}, \bar{s}_t^{\text{due}}, \rho_t^{\text{urg}}, \mu_t^{\text{load}}]$$

where:

- $\bar{q}_t$: average number of jobs currently in the input buffer;
- σ_t^{mach} : standard deviation of remaining processing times across all machines, indicating workload imbalance;
- $\bar{s}_t^{\text{due}}$: average slack of buffered jobs;
- ρ_t^{urg} : proportion of urgent jobs in buffer;
- μ_t^{load} : aggregated estimated processing load per order type across all jobs currently in the system.

All features are normalized. This state formulation enables the Meta-Controller to reason about system-level trade-offs between load balancing, urgency, and processing pressure.

The action space A_t^H consists of a composite tuple:

$$A_t^H = (r_t, d_t, p_t),$$

where $r_t \in [0, 1]$ indicates the proportion of cached jobs to release at time t , $d_t \in \{0, 1\}$ is a binary dispatch flag denoting whether to trigger delivery of completed orders, and $p_t \in \{1, 2, 3\}$ assigns a global priority level for new jobs to guide downstream scheduling.

The reward function is defined to promote long-term scheduling and delivery objectives. Specifically, the immediate reward at time step t is:

$$R_t^H = -\alpha \cdot \text{TWT}_t - \beta \cdot \text{Var}_t,$$

where $\text{TWT}_t = \sum_{i \in \mathcal{O}_t} w_i \cdot \max(0, C_i - D_i)$ is the total weighted tardiness of all active orders, and Var_t is the variance in completion times across machines, penalizing unbalanced workloads. The weights $\alpha, \beta > 0$ control the trade-off between delivery performance and production balance.

The policy is implemented as a Transformer-based deep Q-network $Q(S_t^H, A_t^H; \theta_{high})$, trained using prioritized experience replay and double DQN techniques.

Algorithm 1 Meta-Controller Decision Process

```

1: Initialize Q-network  $\theta_{high}$ , target network  $\theta'_{high}$ , replay buffer  $B_{high}$ 
2: for each episode  $e = 1, \dots, E$  do
3:   Reset environment; observe initial state sequence  $S_0^H$ 
4:   for each decision step  $t = 0, \dots, T_{max}$  do
5:     Encode input  $S_t^H$  using Transformer over recent  $K$  states
6:     Select action  $A_t^H = (r_t, d_t, p_t) \sim \epsilon$ -greedy policy from  $Q(S_t^H, A; \theta_{high})$ 
7:     Release  $r_t$  proportion of cached jobs with priority  $p_t$ 
8:     if  $d_t = 1$  then dispatch completed orders
9:     Observe next state  $S_{t+1}^H$  and reward  $R_t^H$ 
10:    Store transition  $(S_t^H, A_t^H, R_t^H, S_{t+1}^H)$  in  $B_{high}$ 
11:    if update condition met then
12:      Sample minibatch from  $B_{high}$ ; update  $\theta_{high}$  via gradient descent
13:      Update target network  $\theta'_{high} \leftarrow \tau \theta_{high} + (1 - \tau) \theta'_{high}$ 
14:    if termination condition met then break

```

Pseudocode
4.2 Low-Level Actor-Critic Policies

We adopt Dueling DQN for the Meta-Controller and actor-critic GATs for the sub-policies. All agents use prioritized experience replay (PER) for sample efficiency. For training stability, target networks are updated using soft updates:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

where τ is the soft update factor. The training loop proceeds episodically, where each episode simulates a full production-distribution horizon with dynamic job arrivals. At each time step, the Meta-Controller observes global state and issues priority and dispatch actions. The resulting sub-goals guide the warehousing and distribution agents, which schedule jobs and deliveries accordingly. All transitions and rewards are stored and sampled for learning using mini-batch TD loss.

The complete training process is summarized in Algorithm 4.2.

Algorithm 2 HDRL Training Procedure for Warehousing and Distribution Coordination

-
- 1: Initialize high-level meta-policy Q-network θ_{high} , target network θ'_{high} , and replay buffer B_{high}
 - 2: Initialize low-level sub-policy Q-networks $\theta_{low}^w, \theta_{low}^d$ (for warehousing and distribution), target networks $\theta_{low}^{w'}, \theta_{low}^{d'}$, and replay buffers B_{low}^w, B_{low}^d
 - 3: **for** each episode $e = 1, \dots, E$ **do**
 - 4: Reset simulation environment and obtain initial state S_0^{high}, S_0^{low}
 - 5: **for** each time step $t = 0, \dots, T_{max}$ **do**
 - 6: **High-Level Decision:**
 - 7: Select high-level action a_t^{high} using ϵ -greedy policy on $Q(S_t^{high}, a; \theta_{high})$
 - 8: Set sub-goals g_t^w, g_t^d for warehousing and distribution based on a_t^{high}
 - 9: Execute a_t^{high} , observe intermediate reward R_t^{high} and next state S_{t+1}^{high}
 - 10: Store transition $(S_t^{high}, a_t^{high}, R_t^{high}, S_{t+1}^{high})$ in B_{high}
 - 11: **Low-Level Decision (Warehousing):**
 - 12: Select action $a_t^{low,w}$ using ϵ -greedy policy on $Q(S_t^{low,w}, a, g_t^w; \theta_{low}^w)$
 - 13: Execute $a_t^{low,w}$, observe reward $R_t^{low,w}$ and next state $S_{t+1}^{low,w}$
 - 14: Store transition $(S_t^{low,w}, a_t^{low,w}, R_t^{low,w}, S_{t+1}^{low,w}, g_t^w)$ in B_{low}^w
 - 15: **Low-Level Decision (Distribution):**
 - 16: Select action $a_t^{low,d}$ using ϵ -greedy policy on $Q(S_t^{low,d}, a, g_t^d; \theta_{low}^d)$
 - 17: Execute $a_t^{low,d}$, observe reward $R_t^{low,d}$ and next state $S_{t+1}^{low,d}$
 - 18: Store transition $(S_t^{low,d}, a_t^{low,d}, R_t^{low,d}, S_{t+1}^{low,d}, g_t^d)$ in B_{low}^d
 - 19: **Update Networks:**
 - 20: Sample minibatches from $B_{high}, B_{low}^w, B_{low}^d$ using PER
 - 21: Perform gradient descent on $\theta_{high}, \theta_{low}^w, \theta_{low}^d$ to minimize TD loss
 - 22: Periodically soft-update target networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$
 - 23: Update states: $S_t^{high} \leftarrow S_{t+1}^{high}, S_t^{low} \leftarrow S_{t+1}^{low}$
 - 24: **if** episode terminates (e.g., all tasks completed or time limit reached) **then**
 - 25: Break
-